

Documento de Arquitetura — Plataforma de E-commerce (v2)

Revisão baseada no briefing real do cliente. Este documento substitui a análise inicial de "altíssimo tráfego" por uma arquitetura dimensionada corretamente para o volume real informado, evitando over-engineering.

1. Mudança de Premissa (importante)

A análise original foi feita para **100.000+ vendas de altíssimo tráfego**, o que justificava microserviços, Kafka, sharding, etc. O briefing real mostra:

- ~**100 pedidos/dia** no lançamento, com expectativa de **até 500 pedidos/dia** no futuro.
- Isso é um volume **pequeno/médio**, não altíssimo tráfego.

Implicação direta: microserviços, Kafka, sharding de banco e Kubernetes multi-cluster seriam **over-engineering** nesse estágio — trariam custo e complexidade sem benefício real, e atrasariam o lançamento. A recomendação muda para uma arquitetura **enxuta, mas com fronteiras internas limpas**, para permitir evolução futura sem reescrita total caso o volume cresça muito além de 500/dia.

Regra de bolso: 500 pedidos/dia $\approx$ 0,006 pedidos/segundo em média. Mesmo com pico concentrado (ex: todos em 2h), isso está na casa de poucos pedidos por minuto — um único banco relacional bem configurado resolve isso com folga.

2. Modelo de Arquitetura Recomendado

Monólito Modular (Modular Monolith)

Um único deploy, mas com **separação clara de módulos internos por domínio** (Catálogo, Carrinho, Pedido, Estoque, Pagamento, Assinatura, Cupom), cada um com sua própria camada de dados lógica dentro do mesmo banco — preparando o terreno para, no futuro, extrair qualquer módulo como serviço independente se o volume justificar.

Motivo	Detalhe
Custo	Um único serviço para hospedar, monitorar e fazer deploy — infraestrutura muito mais barata
Velocidade	Equipe pequena consegue entregar rápido sem overhead de comunicação entre serviços
Simplicidade transacional	Transações ACID nativas do Postgres resolvem reserva de estoque + pedido + pagamento sem precisar de Saga distribuída
Caminho de evolução	Módulos bem isolados (código e schema) permitem "quebrar" em microserviços depois, só se necessário

Evite por enquanto: Kafka, service mesh, múltiplos bancos de dados por domínio, Kubernetes com múltiplos clusters. Reavaliar isso somente se o volume ultrapassar consistentemente a casa de milhares de pedidos/dia.

3. Modelagem de Domínio (dentro do monólito)

Módulos internos, com fronteiras de código bem definidas (ex: pastas/pacotes separados, sem acesso cruzado direto a tabelas de outro módulo):

1. **Catálogo** — produtos, variações, categorias (catálogo longo)
 2. **Estoque** — quantidade disponível, reserva, sold-out
 3. **Carrinho/Sacola** — itens, cupom aplicado, cálculo de total
 4. **Pedido** — checkout, status do pedido, código de resgate
 5. **Pagamento** — integração com PSP, status de cobrança
 6. **Assinatura** — cobrança recorrente, ciclo de vida (ativa, pausada, cancelada)
 7. **Cupom/Promoção** — regras de desconto
 8. **Cliente** (opcional/pós-compra) — conta criada depois da compra
-

4. Stack Tecnológica Recomendada

Back-end

- **Node.js (NestJS) ou Python (Django/FastAPI)** — ótima produtividade para equipe pequena, ecossistema maduro para e-commerce, e ambos rodam bem em containers simples (sem necessidade de Kubernetes robusto).
- Alternativa: se já houver domínio da equipe em outra stack (Ruby on Rails, Laravel), pode ser igualmente adequada — **a prioridade aqui é velocidade de desenvolvimento**, não performance extrema.

Front-end

- **Next.js (React)** — permite montar as páginas de produto e a sacola rapidamente, com opção de melhorar SEO depois via SSR/ISR, sem exigir design elaborado agora (pode usar componentes prontos, ex: shadcn/ui, Tailwind).

Banco de Dados

- **PostgreSQL único** (gerenciado — ex: RDS, Cloud SQL, Supabase) é suficiente para todo o volume atual e futuro (500/dia).
- **Sem necessidade de réplicas de leitura, sharding ou NoSQL separado** neste estágio — isso só voltaria à mesa se o catálogo crescesse para centenas de milhares de SKUs com buscas complexas (aí sim considerar Elasticsearch/Algolia para busca).
- **Redis** (opcional, mas recomendado): sessão de carrinho para visitante sem conta, e cache simples de disponibilidade de produto.

Infraestrutura

- **Um único ambiente de container** (ex: AWS Fargate, Google Cloud Run, ou até um PaaS como Railway/Render) — evita a complexidade operacional de um cluster Kubernetes completo nesse volume.
 - Auto-scaling simples baseado em CPU/memória (a maioria dos provedores PaaS já resolve isso "de graça").
 - **Fila leve (opcional)**: se necessário desacoplar envio de e-mail/webhook de pagamento, um serviço gerenciado simples (ex: AWS SQS) resolve — não é necessário Kafka/RabbitMQ nesse volume.
-

5. Estoque, Sold-Out e Overselling

Dado o giro alto de estoque e a necessidade de reserva diária, mas **sem** o volume que exigiria Redis + Lua scripts:

- **Optimistic locking** no Postgres (coluna `version` na tabela de estoque) é suficiente para evitar overselling em concorrência simultânea nesse volume.
 - Fluxo recomendado:
 1. Cliente inicia checkout → sistema tenta **reservar** a quantidade dentro de uma transação com `SELECT ... FOR UPDATE` ou checagem otimista.
 2. Se estoque insuficiente → produto marcado como "**Esgotado**" automaticamente (flag `sold_out = true` quando `quantidade <= 0`), removendo/ajustando exibição no catálogo.
 3. Reserva expira automaticamente (ex: 15-30 min) se o pagamento não for concluído, liberando o estoque de volta.
 - Um **job simples agendado** (cron) pode limpar reservas expiradas — não precisa de infraestrutura de mensageria dedicada.
-

6. Pagamentos

Requisitos: cartão, Pix, parcelamento próprio, carteira digital, e possivelmente criptomoeda.

Método	Recomendação
Cartão + Pix + Carteira digital	Um único gateway/adquirente nacional (ex: Pagar.me, Mercado Pago, Stripe se operar fora do Brasil também) cobre os três em uma única integração — reduz complexidade inicial
Parcelamento próprio	Lógica de negócio interna (não depende do gateway): o pedido é criado com um plano de parcelas gerido pela própria plataforma, com cobrança recorrente própria ou via boleto/carnê
Criptomoeda	Recomendo tratar como integração separada e opcional no MVP — gateways como Coinbase Commerce ou BitPay permitem aceitar cripto sem que a loja precise custodiar ativos digitais. Pode ser adicionado numa segunda fase sem impactar o core

Idempotência: toda chamada de cobrança deve usar uma chave de idempotência (evita cobrança duplicada em caso de reenvio de requisição) — isso vale independentemente da escala.

7. Assinatura + Compra Única (modelo híbrido)

O sistema de Pedido precisa suportar dois tipos de item na mesma sacola:

- **Compra única:** fluxo tradicional (pagamento único, geração de código de resgate).
- **Assinatura:** requer um "motor de recorrência" — pode ser:
 - **Usar a recorrência nativa do gateway de pagamento** (a maioria oferece cobrança recorrente por token de cartão) — caminho mais simples e recomendado para o MVP.
 - Construir motor próprio de recorrência apenas se as regras de negócio (pausar, trocar plano, cobrança híbrida) forem muito específicas.

Modelagem sugerida: tabela `subscriptions` vinculada ao cliente (ou ao "pedido-mãe", já que conta é opcional — ver seção 8), com status (`active`, `paused`, `canceled`) e próxima data de cobrança.

8. Checkout Sem Conta Obrigatória (Guest Checkout)

- Cliente completa a compra informando apenas **e-mail/telefone e endereço de entrega**, sem criar senha/conta.
- Pedido gera um **código de resgate único** (ex: hash curto) enviado por e-mail/WhatsApp/SMS, usado para consultar status do pedido ou acessar a assinatura sem login.
- **Pós-compra:** oferecer criação de conta opcional (ex: "quer salvar seus dados para a próxima compra?"), vinculando pedidos anteriores ao e-mail informado, quando a

conta for criada depois.

- Para assinaturas, o código de resgate/token também serve como forma de gerenciar a assinatura (pausar/cancelar) mesmo sem conta formal — importante decidir se isso é aceitável do ponto de vista de segurança, ou se assinatura exigirá conta em algum momento (a definir com o cliente).
-

9. Cupons

- Módulo simples: código, tipo de desconto (percentual ou valor fixo), regras de validade (data, quantidade de usos, valor mínimo do carrinho, produtos elegíveis).
 - Aplicado no momento do cálculo do total da sacola, validado no back-end (nunca confiar em desconto calculado só no front-end).
-

10. Build vs. Buy (revisado para essa escala)

Com volume de 100-500 pedidos/dia, plataformas como VTEX ou commercetools tendem a ser **caras e superdimensionadas** para o estágio atual — o modelo de custo dessas plataformas geralmente é vantajoso a partir de volumes bem maiores.

Recomendação: desenvolvimento **proprietário enxuto** (monólito modular descrito acima) é o caminho de menor custo e maior flexibilidade agora, especialmente porque:

- O modelo híbrido de assinatura + compra única raramente é bem suportado "out of the box" em plataformas de e-commerce tradicionais.
- O checkout sem conta obrigatória, com resgate por código, também é uma customização que exigiria contornar bastante o padrão da maioria das plataformas prontas.

Reavaliar plataformas prontas apenas se, no futuro, o volume crescer para uma escala onde manter o time interno de desenvolvimento se torne mais caro que o licenciamento.

11. Roadmap de Evolução (quando revisar essa arquitetura)

Gatilho	Ação recomendada
Catálogo passa de ~50.000 SKUs com buscas lentas	Adicionar Elasticsearch/Algolia só para busca
Volume ultrapassa ~2.000-3.000 pedidos/dia de forma sustentada	Avaliar extrair módulo de Pedido/Estoque como serviço separado
Picos de tráfego começam a gerar lentidão perceptível	Adicionar Redis para cache de catálogo e CDN para páginas estáticas
Volume de e-mails/webhooks cresce muito	Introduzir fila assíncrona leve (SQS)
Necessidade de reconciliação financeira complexa entre assinatura e compra única	Considerar plataforma de billing dedicada (ex: Stripe Billing)

12. Resumo da Recomendação

- **Arquitetura:** Monólito modular, com módulos internos bem separados por domínio.
- **Stack:** Next.js (front) + Node.js/NestJS ou Python/FastAPI (back) + PostgreSQL + Redis (opcional).
- **Infra:** Container único gerenciado (Cloud Run/Fargate/PaaS), sem Kubernetes complexo.
- **Estoque:** Optimistic locking no Postgres + flag de sold-out automática.
- **Pagamento:** Um gateway nacional cobrindo cartão/Pix/carteira + módulo próprio de parcelamento + integração de cripto isolada e opcional.
- **Assinatura:** Usar recorrência nativa do gateway sempre que possível.
- **Checkout:** Sem conta obrigatória, com código de resgate; conta opcional pós-compra.
- **Build vs Buy:** Proprietário enxuto — plataformas prontas ficariam caras e pouco flexíveis para o modelo híbrido de assinatura + compra única sem conta.

Este desenho evita gastar tempo e dinheiro com complexidade que só faria sentido em uma escala 100x-1000x maior, mas mantém as fronteiras de código organizadas o suficiente para crescer sem reescrever tudo do zero.